

Trabajo Práctico 1 — Desarrollo de Aplicaciones Móviles

Cátedra: Arquitecturas Móviles — UTN Facultad Regional San Francisco

Profesor: Ing. Juan Pablo Bono

Alumno: Fabricio Quinteros

Aplicación: Tarjetazo — gastos de una tarjeta de crédito compartida

Repositorio: <https://github.com/FabriQuinteros/tp1-arquitecturas-moviles>

1. Problema

Cuando varias personas usan la misma tarjeta de crédito, el titular es el único que ve el resumen y el único que paga. A fin de mes tiene que reconstruir de memoria cuánto le debe cada uno.

Tarjetazo registra cada gasto en el momento en que ocurre, con su división entre personas, y muestra cuánto debe cada una.

2. Alcance

Esta entrega es una aplicación Android nativa que funciona contra el almacenamiento del propio dispositivo, sin servidor. Registrar gastos, dividirlos y ver la deuda resultante.

Queda afuera todo lo que exija backend —cuentas de usuario, invitar colaboradores, sincronizar entre dispositivos, aprobar gastos de terceros— y también las variantes que no agregan nada al objetivo del trabajo práctico: gastos en dólares, cuotas, impuesto de sello y categorías. El objetivo acá es el ciclo de vida de las Activities y la comunicación entre componentes, no el dominio financiero completo.

3. Requerimientos funcionales

ID	Requerimiento
RF-1	El usuario registra un gasto con comercio e importe; la fecha se toma automáticamente del día de la carga.
RF-2	El usuario divide un gasto entre varias personas asignando un porcentaje a cada una; la aplicación no permite guardar una división que no sume cien por ciento.
RF-3	Un gasto sin división explícita se imputa entero al titular.
RF-4	La pantalla principal lista los gastos registrados, con el total.
RF-5	El usuario elimina un gasto, con una confirmación previa.
RF-6	La aplicación muestra cuánto debe cada persona, sumando su parte de cada gasto.

ID	Requerimiento
RF-7	El usuario comparte el resumen de deuda hacia otra aplicación del dispositivo.
RF-8	Si el usuario abandona la aplicación con un gasto a medio cargar, el borrador se conserva y se restaura al volver.
RF-9	Los gastos quedan guardados entre ejecuciones: cerrar la aplicación no los pierde.

4. Requerimientos no funcionales

ID	Categoría	Requerimiento
RNF-1	Compatibilidad	Funciona desde Android 8.0 (API 26) en adelante.
RNF-2	Disponibilidad	Opera sin conexión a internet.
RNF-3	Exactitud	Los importes no se calculan con punto flotante binario: dividir un gasto entre tres personas descuadra los centavos. El resto de la división se asigna de forma explícita a una de las partes.
RNF-4	Robustez	Rotar la pantalla o recibir una llamada en medio de la carga de un gasto no pierde lo cargado.
RNF-5	Privacidad	Los datos no salen del dispositivo salvo que el usuario use explícitamente la acción de compartir. La aplicación no declara ningún permiso.

5. Limitaciones propias de la plataforma móvil

Son las que condicionan el diseño, y lo que la guía del trabajo práctico pide identificar:

- **El sistema operativo puede destruir la aplicación en cualquier momento.** Android libera memoria cerrando procesos en segundo plano: el usuario que sale a mirar un mensaje puede volver a una aplicación que se reinició. El estado no puede vivir solo en memoria, y de ahí salen RF-8 y RNF-4.
- **La pantalla rota durante la ejecución,** lo que destruye y vuelve a crear la Activity.
- **La conectividad es intermitente:** el gasto se carga parado en la caja de un comercio. De ahí RNF-2.
- **La entrada de datos es cara:** escribir en un teclado táctil, parado, es lento. Por eso el gasto tiene tres campos y la división es opcional (RF-3).

6. Arquitectura de la aplicación

6.1 Componentes

Componente	Tipo	Responsabilidad
MainActivity	Activity	Lista los gastos con el total y ofrece registrar uno nuevo o ver las deudas.
GastoEditorActivity	Activity	Alta de un gasto y su división por persona.
DeudasActivity	Activity	Deuda por persona, con la acción de compartirla.

Componente	Tipo	Responsabilidad
Repositorio	Clase de datos	Único punto de lectura y escritura del almacenamiento local.
Dominio	Funciones de dominio	Reparto de importes entre personas. Sin dependencias de Android, para poder probarlo sin emulador.

6.2 Navegación e Intents

- **Intents explícitos:** `MainActivity` → `GastoEditorActivity`, que devuelve el gasto cargado como resultado a la Activity que la lanzó, y `MainActivity` → `DeudasActivity`.
- **Intent implícito:** `ACTION_SEND` para compartir el resumen de deuda. La aplicación declara la intención y el sistema resuelve qué aplicación la atiende —mensajería, correo, notas— sin que Tarjetazo sepa cuál está instalada.

6.3 Declaraciones del Android Manifest

- Las tres Activities, con `MainActivity` marcada como punto de entrada (`LAUNCHER`).
- Versión mínima de Android declarada según RNF-1.
- Copia de respaldo automática desactivada (`android:allowBackup="false"`), para que los datos no salgan del dispositivo sin que el usuario lo decida (RNF-5).
- Ningún permiso: la aplicación no necesita internet, ni ubicación, ni cámara, ni contactos. Es verificable en la ficha de la aplicación antes de instalarla.

6.4 Ciclo de vida

Es el punto central del trabajo práctico. La carga de un gasto puede quedar a mitad de camino, y `GastoEditorActivity` responde así:

Evento	Qué hace la aplicación
<code>onCreate</code>	Si hay estado guardado —por ejemplo, después de rotar— los campos ya vienen restaurados por el sistema. Si no lo hay, carga el borrador persistido.
<code>onSaveInstanceState</code>	El sistema guarda el texto de cada campo antes de destruir la Activity por una rotación o por falta de memoria.
<code>onPause</code>	Persiste el borrador en el almacenamiento local: es el último momento garantizado antes de que el sistema pueda terminar el proceso. Si el gasto ya se guardó o se canceló, lo vacía.

`GastoEditorActivity` se lanza esperando un resultado: `MainActivity` sigue viva debajo en la pila y se detiene (`onStop`) en lugar de destruirse. Es donde se ve la diferencia entre las dos cosas.

7. Stack tecnológico

Capa	Elección	Motivo
Lenguaje	Kotlin	Lenguaje oficial de Android.
Entorno	Android Studio	Requerido por la guía del trabajo práctico.

Capa	Elección	Motivo
Interfaz	Vistas XML con <code>ListView</code>	Componentes nativos de la plataforma, sin dependencias extra. Hace explícito el ciclo de vida de la Activity, que es lo que hay que demostrar.
Importes	Aritmética decimal de precisión arbitraria	Exigido por RNF-3.
Persistencia	<code>SharedPreferences</code> con JSON	Incluido en Android: alcanza para los gastos de una tarjeta y cubre RF-9 sin agregar dependencias.
Construcción	Gradle	Estándar del ecosistema.

8. Criterios de aceptación

1. Se carga un gasto de 10.000,01 pesos dividido en 50 % y 50 % entre dos personas, y la aplicación muestra 5.000,01 para la primera y 5.000,00 para la segunda: el centavo que sobra tiene dueño y el total no se descuadra.
2. Con un gasto a medio cargar, se rota el dispositivo y no se pierde ningún campo.
3. Se cierra la aplicación por completo y al volver a abrirla están todos los gastos y las deudas.
4. La acción de compartir abre el selector del sistema con el resumen de deuda en texto.
5. La aplicación se instala y opera con el modo avión activado, y su ficha en el sistema no declara ningún permiso.

9. Implementación

El código fuente está en <https://github.com/FabriQuinteros/tp1-arquitecturas-moviles>.

Archivo	Qué contiene
<code>AndroidManifest.xml</code>	Las tres Activities, <code>MainActivity</code> como punto de entrada, ningún permiso y sin copia de respaldo.
<code>MainActivity.kt</code>	Lista de gastos y total. Lanza el editor esperando resultado (intent explícito) y guarda el gasto que devuelve.
<code>GastoEditorActivity.kt</code>	Carga del gasto y manejo del borrador a lo largo del ciclo de vida.
<code>DeudasActivity.kt</code>	Deuda por persona y acción de compartir (intent implícito <code>ACTION_SEND</code>).
<code>Dominio.kt</code>	Lectura de importes, validación de la división y reparto de deudas, sin dependencias de Android.
<code>Repositorio.kt</code>	Lectura y escritura de gastos y borrador en <code>SharedPreferences</code> .
<code>ActividadRegistrada.kt</code>	Clase base de las tres Activities: escribe en Logcat cada evento del ciclo de vida.
<code>DominioTest.kt</code>	Pruebas unitarias del reparto: centavo sobrante, porcentajes que no suman cien, personas repetidas e importes.

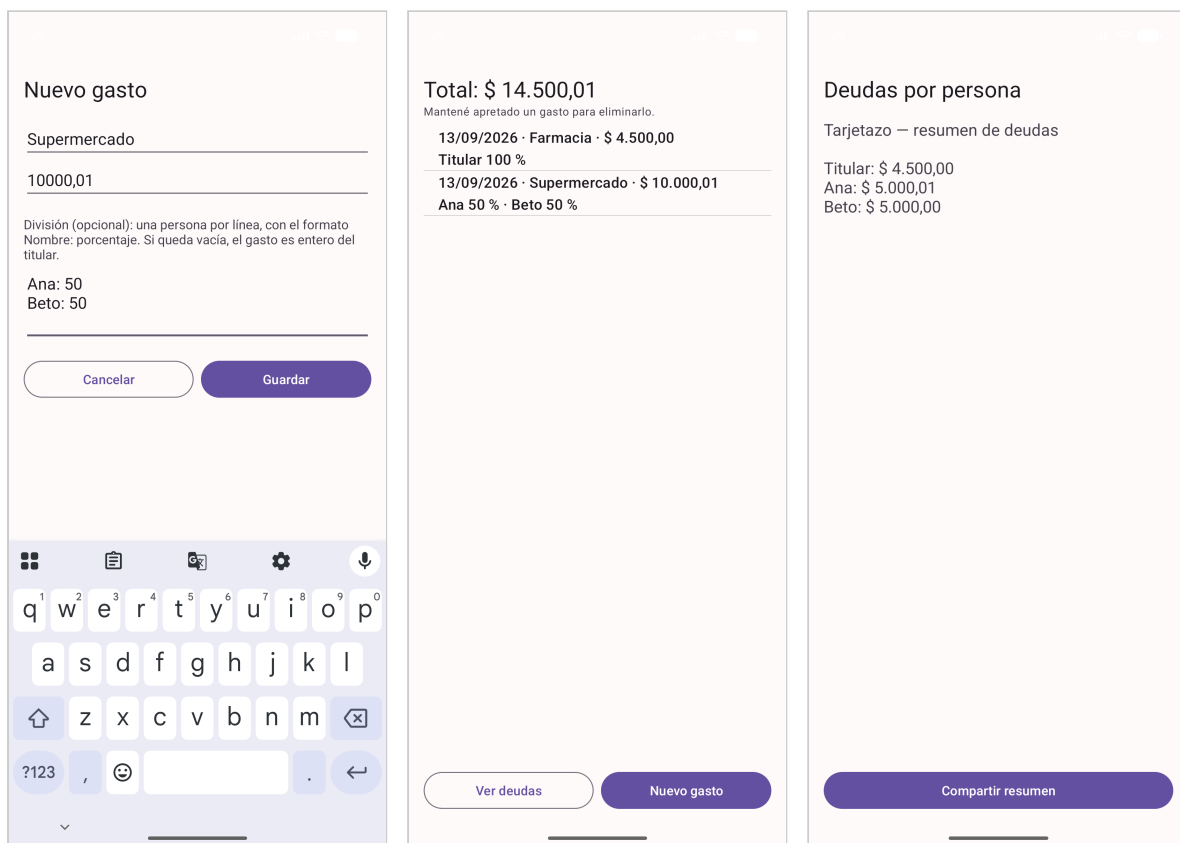
9.1 Cómo observar el ciclo de vida

En Android Studio, abrir **Logcat** y filtrar por `tag:Ciclo`. Cada Activity registra cada evento que recibe. Tres recorridos muestran lo central del trabajo práctico:

- **Abrir el editor desde la lista:** MainActivity pasa por onPause y onStop, pero no por onDestroy: queda detenida debajo en la pila. Al volver, pasa por onStart y onResume.
- **Rotar el dispositivo con un gasto a medio cargar:** GastoEditorActivity pasa por onPause, onStop, onSaveInstanceState y onDestroy, y enseguida por un nuevo onCreate (estado guardado: true). Los campos conservan lo escrito.
- **Salir de la aplicación a mitad de carga y cerrarla desde recientes:** al volver a abrir el editor aparece onCreate (estado guardado: false) y el borrador está cargado igual, porque se persistió en onPause.

9.2 Evidencia de ejecución

Capturas del emulador (Pixel 7, Android 17). Se cargó un gasto de 10.000,01 pesos dividido en 50 % y 50 % entre Ana y Beto, y otro de 4.500 pesos sin división. La pantalla de deudas muestra el centavo sobrante asignado a Ana, como pide el criterio de aceptación 1.



Editor rotado con un gasto a medio cargar: los campos conservan lo escrito.

Nuevo gasto

Kiosco

1200

División (opcional): una persona por línea, con el formato Nombre: porcentaje. Si queda vacía, el gasto es entero del titular.

Ana: 50

Beto: 50

Cancelar

Guardar

Registro de Logcat (filtro `tag:Ciclo`) de ese tramo: desde la lista se abre el editor, se carga el gasto a medio camino, se rota a horizontal, se vuelve a vertical y se cancela. Cada rotación pasa por `onSaveInstanceState` y `onDestroy`, y la Activity se vuelve a crear con `onCreate` (estado guardado: `true`). `MainActivity` nunca se destruye: queda detenida y vuelve con `onRestart`.

```

MainActivity.onPause
GastoEditorActivity.onCreate (estado guardado: false)
GastoEditorActivity.onStart
GastoEditorActivity.onResume
MainActivity.onStop
MainActivity.onSaveInstanceState
GastoEditorActivity.onPause
GastoEditorActivity.onStop
GastoEditorActivity.onSaveInstanceState
GastoEditorActivity.onDestroy
GastoEditorActivity.onCreate (estado guardado: true)
GastoEditorActivity.onStart
GastoEditorActivity.onResume
GastoEditorActivity.onPause
GastoEditorActivity.onStop
GastoEditorActivity.onSaveInstanceState
GastoEditorActivity.onDestroy
GastoEditorActivity.onCreate (estado guardado: true)
GastoEditorActivity.onStart
GastoEditorActivity.onResume
GastoEditorActivity.onPause
MainActivity.onRestart
MainActivity.onStart
MainActivity.onResume
GastoEditorActivity.onStop
GastoEditorActivity.onDestroy

```